

Vibe Coders Final Release Deliverables

Final Release Test Results

The following is a summary of the Beta Test results conducted between April 17th and April 23rd. The people who tested our product consisted of roommates, friends, fraternity members, and more. These tests were completed using the UAT method (User Acceptance Testing), where they were given tasks that needed to be completed without being explicit instructions on how to complete them. Testers were able to provide us feedback after every test, which would be beneficial in helping us improve our project. The full test results will be after the contribution summary at the very end.

Test Case Number	Feedback
TC 1-4 (Account Login/Creation)	Based on the provided feedback, the login and account creation system was functional and worked as the user expected. Some minor feedback was improvements we could include in the future, such as a password strength indicator, adding special characters in the password for security, and a way to view the password input.
TC 5-6 (Route Optimization)	These tests received a lot of feedback regarding the route optimization. Much of the feedback

Test Case Number	Feedback
	concerns how route optimization works, as it was implemented using aisle ordering. Future recommendations included talking about potential pathfinding algorithms, such as Dijkstra's algorithm, A*, and more. Using these algorithms would be more beneficial than just ordering the items by aisle number.
TC 7-8 (ZipCode Validation)	The primary feedback relating to this test was that there was only one valid zipcode, the Troy zipcode. Recommendations involved incorporating the Google Maps API in the future to obtain more grocery locations, as opposed to just the ones in Troy. Also, something else that was brought up was if invalid zipcodes were entered, there should be a pop-up explaining that the inputted zipcode isn't valid.
TC 9 (Cost Optimization)	The response to this test was mostly positive, as the cost optimization feature worked as expected and provided the testers with the cheapest option. Something that was noted was that the total cost

Test Case Number	Feedback
	was shown only at the top, so when users would scroll to see all possible grocery options, they wouldn't see the total cost. They recommended adding the total cost in a place that can always be viewed.
TC 10-12 (Accessibility Settings)	The feedback regarding changing accessibility settings was positive, as most people found the inclusion of dark mode to be great, as they typically have dark mode on their phones. Changing between dark mode and light mode was fluid, and adjusting the text size worked as expected. A future improvement would be adding a slider to adjust the text size as opposed to just increasing the text size and adjusting the light/dark mode to incorporate more colors as a way to set a theme for the profile, as opposed to just light/dark mode.

Best Practices Evidence

We, as a team, have implemented all required best practices as well as three additional ones, and we will have evidence to support this. The team utilized a Jira space as the project management website, as suggested by the tools page from the course website. The Grocery

Finder space has been shared with the Professor(sddteaminstructor@gmail.com), and the link is here <https://alanlee.atlassian.net/jira/software/projects/SCRUM/summary>. For the second mandatory best practice, which is a website for tracking progress and specific tasks, we utilized the same Jira project space as the first mandatory best practice. For the third mandatory best practice, the team created a git repository https://github.com/Alan-Lee7/SDD_Grocery_Pathfinding. This repository contains all of the team's code for this semester. The fourth mandatory best practice we implemented is our coding standards: we officially adopted PEP 8 (<https://peps.python.org/pep-0008/>) as our foundational coding standard, alongside PEP 257 for docstring conventions (<https://peps.python.org/pep-0257/>). Our team also follows 3 specific rules regarding naming conventions: Every name must clearly state what the variable or class represents, use an underscore as the first character to a helper or private function, and no underscores for class names. Finally, for the mandatory coding practice, we implemented several object-oriented design patterns. For abstraction: in backend/services/interfaces.py through Protocols like ProductMatcher and AislePlanner, which define essential methods (match and plan) so that components like optimize_service.py can interact purely with what a service does, rather than worrying about how it does it. For polymorphism: in backend/services/optimize_services.py, where the variable matcher can be different concrete classes, such as DefaultMatcher, StoreBrandPreferredMatcher, BudgetAwareMatcher, or CouponAwareMatcher, and all implementations can call the same method match. For inheritances: in backend/factories/matcher_factory.py and planner_factory.py, where MatcherFactory is a base class, BasicMatcherFactory, PreferenceMatcherFactory, BudgetMatcherFactory, and AdvancedMatcherFactory inherit and override the create_matcher function with their own logic. Finally, for encapsulation: in

backend/models.py, with this line `ShoppingCart.total_cost(self)`, it is encapsulated because external code does not need to know how prices are computed within the class, and all internal pricing logic stays within the `ShoppingCart` class.

The three additional best practices used are: design pattern implementation, third-party component(npm ecosystem), and build tools. The design pattern implemented is the factory design pattern, at `backend/factories/matcher_factory.py` and `backend/factories/planner_factory.py`. These two files implement the factory design pattern, where instead of hardcoding the instantiation of specific classes, we delegate this responsibility to factory classes. This allows our system to dynamically decide which algorithm implementation to use at runtime based on the user's context, without modifying the core. The third-party component used is npm. Our application utilizes a split-stack architecture, with backend dependencies built using Python/Flask. The current product relies on npm as the third-party component manager for our frontend ecosystem because, rather than building complex UI state management or asset bundling pipelines from scratch, npm allows us to securely integrate robust, vetted open-source frameworks. Finally, for the build tools, we have created a Makefile and use Make as our automated build tool. Rather than relying on simple scripts, the Makefile acts as an intelligent orchestrator for our frontend toolchain.

Steps For Deployment:

First, to deploy Grocery Path Finder, the user needs to create an account on the site. For this example, we will be using Vercel for the frontend. The user needs to link their GitHub account to their Vercel account so Vercel can access the repo. Then they need to import the repo into a project. Vercel will autodetect that it is a Vite app. Double-check that the settings are as

follows: Framework Preset = Vite, Build Command = pnpm run build, Output Directory = dist.

After that, hit deploy, and you should have a URL that says your-project.vercel.app.

Next, for the backend, we will use Render, but other sites will work too. The user needs to create a new Web Service and link the repo to it. They also need to set the root directory to backend and configure the settings as follows: Runtime = Python, Build Command = pip install -r requirements.txt, Start Command = flask run --host=0.0.0.0 --port=10000, and click deploy.

This will give us a URL like <https://your-app.onrender.com>.

Now, back to Vercel, we need to set the backend URL as an env variable. Copy the URL render given to us and create this variable in Vercel as VITE_API_URL=<https://your-app.onrender.com>. Then we need to update src/app/api.ts and add this line, “const API_BASE = import.meta.env.VITE_API_URL + "/api";”. Finally, we need to allow the frontend on the backend using CORS. In the Flask app, we need to configure CORS to allow requests from Vercel URL using this line, “CORS(app, origins=["https://your-app.vercel.app"])”.

Project Status Report

The project has come to the point of full deployment following the completion of key features and fixing bugs found in our project during beta testing. During the week between the Beta release during class and our presentation we received feedback regarding our accessibility settings, coupon logic, cost optimization, and more. In terms of accessibility settings, we should implement the dark/light mode feature, but during testing we found a minor issue with it. There was an occasional flash of a white screen when we should be in dark mode. This was an issue with how we were handling the swap between the two colors and would be fixed by Alan before the final release. Another issue with the accessibility settings was changing the font size. At some points there would be an overflow of characters. This issue would be fixed by Will. There

was another issue with the filter feature when filtering between types of grocery items. Items were given specific tags based on what type of item they were, but when we attempted to filter by a specific category, we would obtain items that did not belong in said category. There was an issue with our sorting logic which would be fixed by Sebastian. An issue relating to the couponing logic was that when we attempted to compare stores, the coupons wouldn't be properly applied for comparison. There was an issue with how we were handling coupons between the initial state of a chosen grocery store and comparing stores. This issue would be fixed by Peijing. All these issues would be fixed prior to the final presentation and we were able to present a functional project that didn't have the aforementioned bugs. There aren't any outstanding issues with the current state of the project but there could be many improvements made to our features. As this is the final deliverable, the team wants to thank the SD&D TAs and Prof. Sturman for teaching and guiding us throughout this project. We all found the skills taught to us to be applicable to our future careers and learning more about softwares that is used in professional settings was useful as well. Thank you for the time and dedication, from the Team of Vibe Coders.

Contribution Summary

Alan Lee

Between the Beta release and the final presentation, I worked on fixing the dark/light mode switching, as there was an occasional issue with it where the screen would momentarily turn white before turning back to dark when the dark mode feature was used. I also helped with some of the beta testing and created the summary for the final test results. I worked on the project status report and set up meetings with the group to discuss the final presentation and set up times to practice it online and in person. I also help work on the presentation by cleaning up

and working on some slides.

Peijing Xu

Between the Beta release and the final presentation, I worked on clearing up the code, making clean updates to the coupon logic. I helped with constructing the beta testing and distributing those across to thirty people that I know. I then worked on the best practice evidence section of the final release deliverables. I also participated in all meetings with the group pertaining to the final deliverable as well as the final presentation. I also helped working on the presentation slides by writing out the feature introductions and cleaned up some slides as well.

Sebastian Grammas

Between the Beta release and the final presentation, I worked on cleaning up a few bugs, along with implementing a feature for users to be able to easily delete items and edit the cart to add more of a specific item in a centralized location. I also wrote out the deployment steps for the Final Release. Along with that, I helped create the slide deck and made the template for what slides we planned on using, and wrote out the slides that I presented.

Qizheng Sui

Between the Beta release and the final presentation, I worked on my assigned features, refining functionality and fixing issues to ensure the system met requirements. I also helped with testing by reviewing results and making adjustments based on feedback. Additionally, I contributed to the final deliverables by organizing and polishing content, participated in team meetings, and assisted with the presentation by writing and revising slides.

Grading rubric

[FinalReleaseRubric.pdf](#)

Beta Test Results

Test Description	Tester	Completion Status	Comments
User Test 1: Valid Login	Alan Lee	Completed	Successfully logged in with credentials.
User Test 2: Invalid Login	Alan Lee	Completed	The system correctly rejected invalid credentials and prompted the user to try again.
User Test 3: Valid Registration	Alan Lee	Completed	The user can create a new account by inputting their email and password.
User Test 4: Invalid Registration	Alan Lee	Completed	The system correctly failed the registration process for a pre-existing email.
User Test 5: Optimized Route (Multiple Items)	Alan Lee	Completed	The system generated a route based on the mock data, and the shopping cart is displayed in a readable order.

Test Description	Tester	Completion Status	Comments
User Test 6: Optimized Route (Single Item)	Alan Lee	Completed with Issues	Completed the route for a single item and closed the session.
User Test 7: Store Selection by Valid ZIP	Alan Lee	Completed	Nearby stores shown based on inputted zipcode.
User Test 8: Store Selection by Invalid ZIP	Alan Lee	Completed with Issues	The system was able to properly handle an invalid zip code entry.
User Test 9: Optimize Costs (Compare Stores)	Alan Lee	Completed	The program successfully suggested an accurate split shopping plan between locations.
User Test 10: Settings (Large Font)	Alan Lee	Completed	The website successfully changed the font size to a larger size across the interface.
User Test 11: Settings (Dark Mode Toggle)	Alan Lee	Completed	The user was successfully able to toggle dark mode on.

Test Description	Tester	Completion Status	Comments
User Test 12: Settings (Dark Mode Persistence)	Alan Lee	Completed	Dark mode preference persisted correctly after reopening.